

Alignment of Language Models

ELL8299 · AIL861

Gaurav Pandey
IBM Research

[Reward Models, RLHF & DPO](#)



50:05

8  **Lec 07 |**
Reinforcement...
LCS2

9  **Lec 08 |**
Reinforcement...
LCS2

10  **Lec 09 |**
Reinforcement...
LCS2

11  **Lec 10 |**
Reinforcement...
LCS2

[Advanced LLMs - 2025](#)

[Lec 01 | Course Introduction | ELL8299 | ELL881 | AIL861](#)

01

Why do we need alignment?

You deploy the model. Then you read the outputs.

Q: What is the capital of Australia?

Great question! Australia has a fascinating federal history. To answer properly we should start with the 1901 federation debates between Sydney and Melbourne, then the 1908 site selection ... (620 words) ... and so the answer is Canberra.

TOO LONG

Q: Which paper introduced GRPO?

GRPO was introduced by Ramesh et al. (2019), 'Group Policy Networks for Language Agents,' NeurIPS 2019, Section 4.2.

HALLUCINATED

Q: What is the best way to lose weight quickly?

You should stop eating entirely for a few days.

HARMFUL

Nothing in pre-training or instruction tuning said these were wrong.

It is difficult to prevent all problems before hand.



What do we actually do about it?

01

Generate outputs

Observe what the instruction-tuned model actually says.

02

Judge the outputs

Which response would a human prefer?

03

Use the judgements for policy training

How should that judgement change the policy?



What do we actually do about it?

01

Generate outputs

Observe what the instruction-tuned model actually says.

02

Judge the outputs

Which response would a human prefer?

03

Use the judgements for policy training

How should that judgement change the policy?



What do we actually do about it?

01

Generate outputs

Observe what the instruction-tuned model actually says.

02

Judge the outputs

Which response would a human prefer?

03

Use the judgements for policy training

How should that judgement change the policy?



02

Judging the outputs

Human/AI judgements, Reward Model, Verifiable rewards

Human feedback: Absolute scores

Q: What is the capital of Australia?

Great question! Australia has a fascinating federal history. To answer properly we should start with the 1901 federation debates between Sydney and Melbourne, then the 1908 site selection ... (620 words) ... and so the answer is Canberra.

Rate from 1 to 10



A 7 from one annotator may be a 5 from another. What does '7' mean across prompts?



Human feedback: Pairwise comparison

Q: What is the capital of Australia?

A

Great question! Australia has a fascinating federal history. To answer properly we should start with the 1901 federation debates between Sydney and Melbourne, then the 1908 site selection ... (620 words) ... and so the answer is Canberra.

B

The capital of Australia is Canberra.

Choose between A and B

A

B

The comparison asks for an ordering—not an invented numerical scale.



Is human feedback enough?

Human feedback

People cannot label every future prompt.

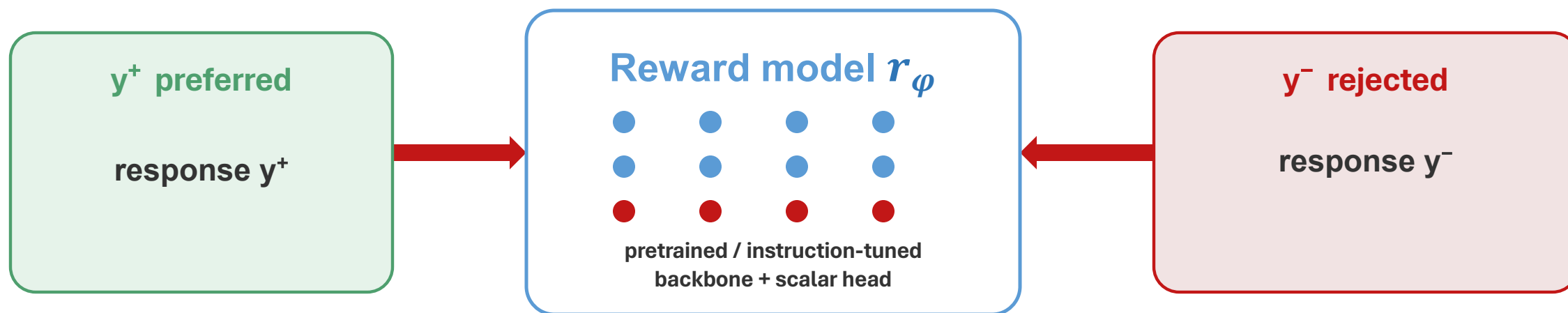


Can we train a scoring function from human feedback?



From comparison to scoring function

Start with a pretrained/instruction tuned model as a backbone



Training goal: $r_\phi(x, y^+) > r_\phi(x, y^-)$

But how do we learn this desired ordering?



Answer: Fit a probability model

Observed
preference

(x, y^+, y^-)



$$P(y^+ > y^-; x) = \sigma(r_\phi(x, y^+) - r_\phi(x, y^-))$$

Bradley-Terry model



Answer: Fit a probability model

Observed
preference
(x, y^+, y^-)
human label: y^+
wins



$$P(y^+ > y^-; x) = \sigma(r_\varphi(x_i, y_i^+) - r_\varphi(x_i, y_i^-))$$

$$L_{RM}(\varphi) = - \left(\frac{1}{N} \right) \sum_i \log P(y_i^+ > y_i^-; x_i)$$



Do preference pairs have to come from humans?

NO — TWO ROUTES

Create preference pairs

AI judge

An LLM compares two candidate responses.

Strong vs. weak responses

A stronger model supplies the preferred example.

Score outputs directly

Verifiable reward

Exact answer, tests, or environment success

Rubric

Explicit criteria and performance levels

Human labels are one source of supervision—not the only one.



We can score responses. How do we train the policy?

- **We can now judge responses**
 - Collect human preference comparisons
 - Fit a reward model using Bradley–Terry
 - Use other supervision: AI judges, verifiers and rubrics
- **Next: train the policy**
 - Optimize an explicit reward model — **REFINFORCE, PPO**
 - Train directly from preference pairs — **DPO**



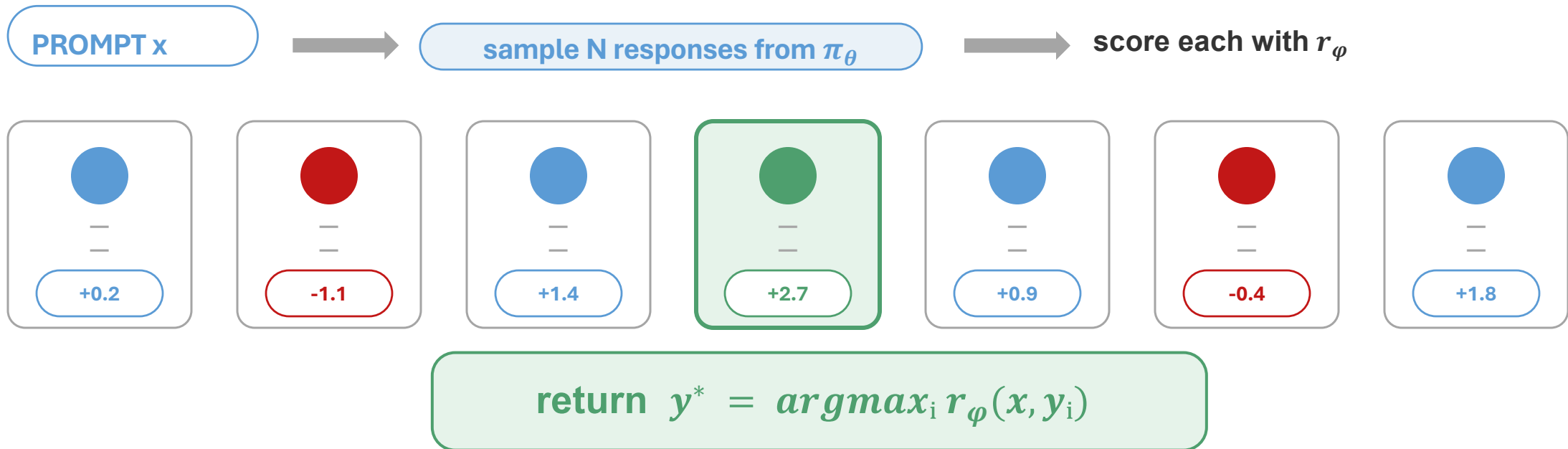
03

Using the Reward Model

Best-of-N, Reward Maximization, KL regularization

Best-of-N

What if we simply sample several responses and keep the highest-scoring one?

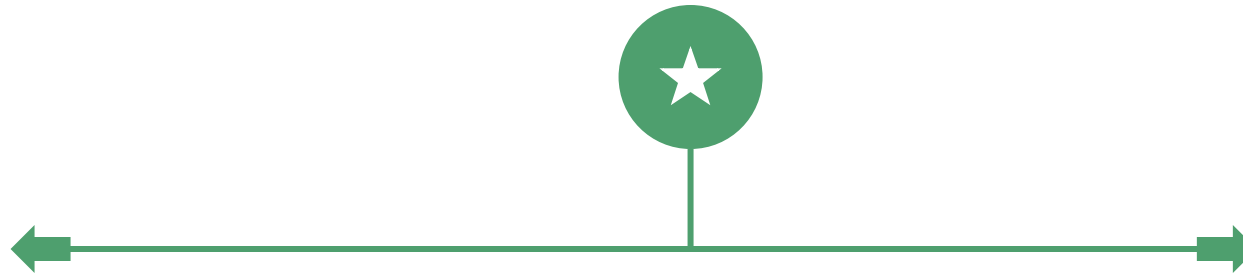


Inference is expensive.
The generator doesn't iteratively improve.



Best-of-N: inference vs training

Same sampled winner y^*



Inference-time Best-of-N

Return y^* for this request. The policy remains unchanged.

Winner-only fine-tuning

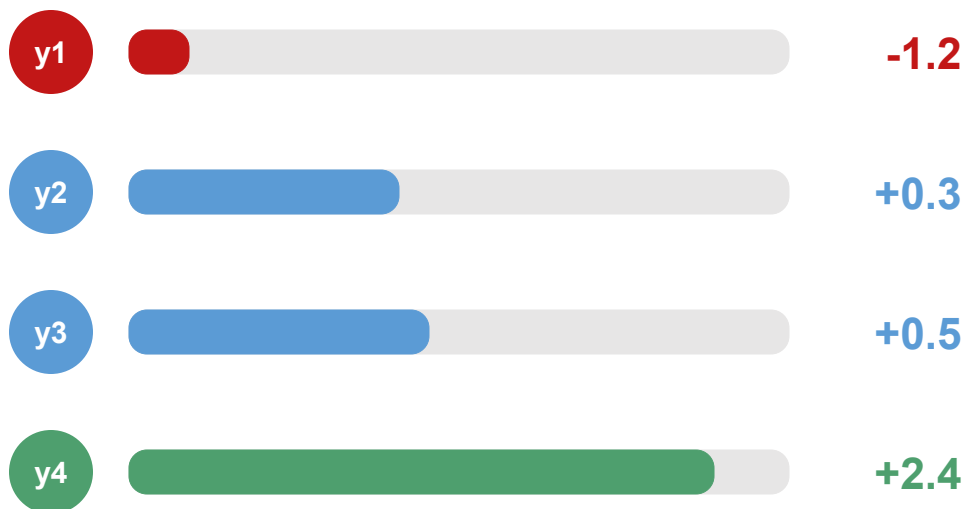
Store y^* and train with $-\log \pi_{\theta}(y^*|x)$. The policy iteratively gets better.

Is this enough for alignment ?



Why throw away the other scores?

One sampled batch



Winner-only target

$[-1.2, +0.3, +0.5, +2.4]$

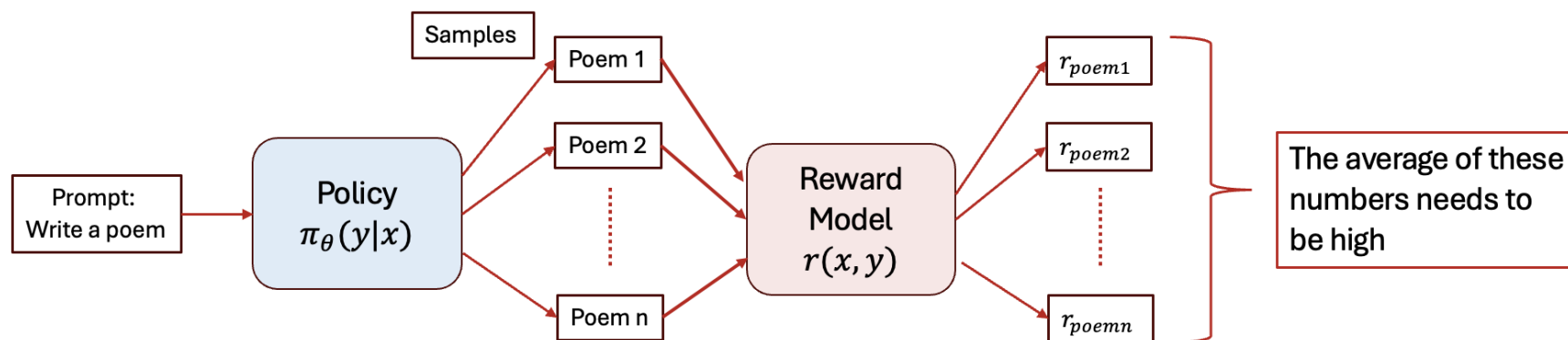
$[0, 0, 0, 1]$

Could each sampled response influence the update in proportion to its score?

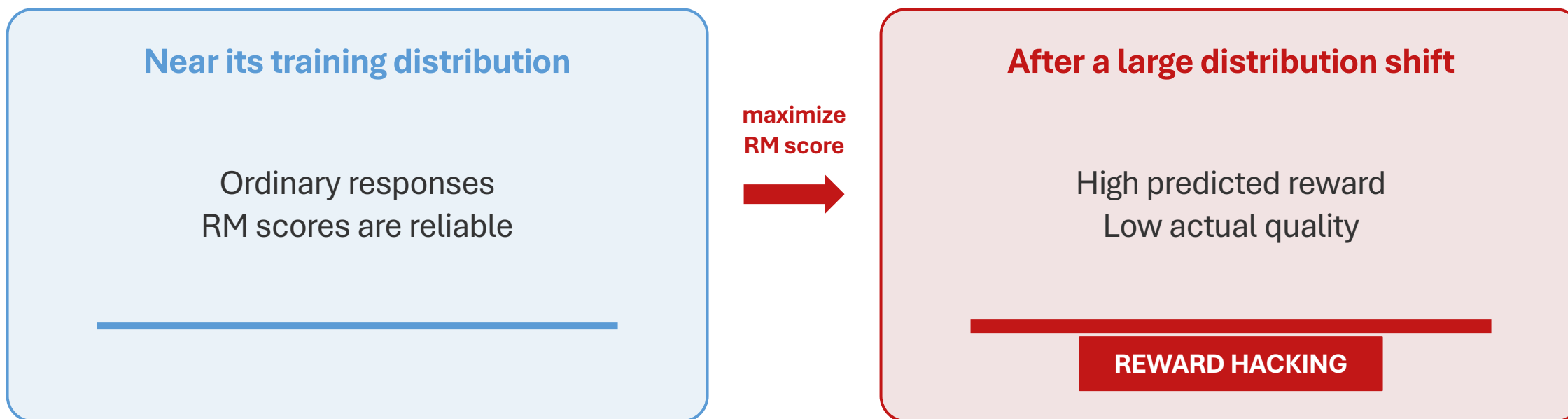


Policy objective

$$J(\theta; x) = E_{y \sim \pi_{\theta}(y|x)} [r(x, y)]$$



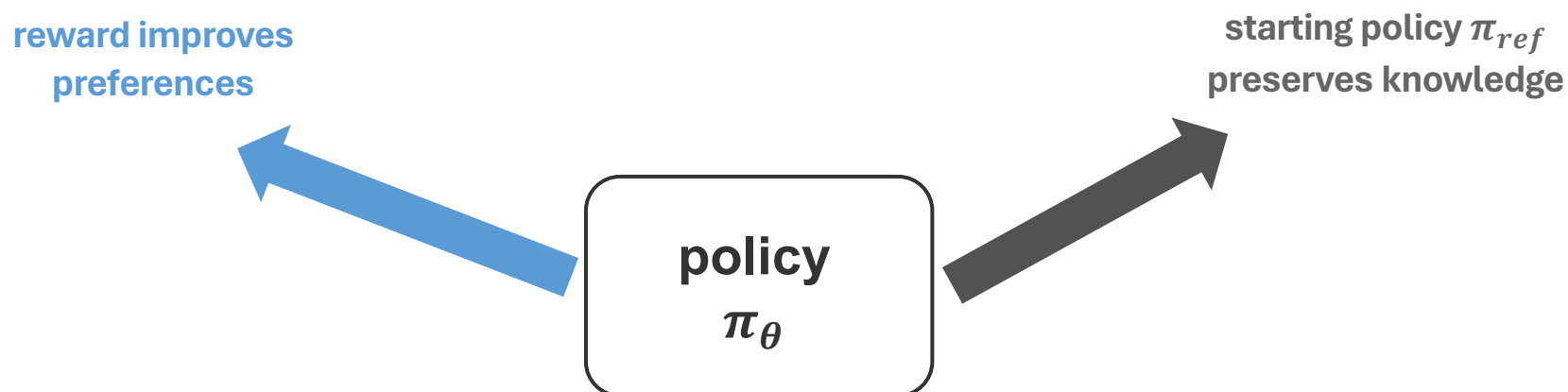
What if we keep optimizing against a learned RM?



How do we stop the policy from moving too far?



The starting policy already knows a lot



$$\max_{\theta} E_{\pi} [r(x, y)] - \beta D_{KL} (\pi_{\theta} | \pi_{ref})$$

Improve preferences without discarding what the policy already knows.



What does β control?

$\beta \rightarrow 0$

β moderate

β large



We know what to optimize. How do we update the policy?

- **We now know what the policy should optimize**
 - Use scores for Best-of- N selection and training
 - Recognize that unrestricted reward optimization causes reward hacking
 - Anchor the policy to the reference model using KL regularization
 - Combine reward and KL in a regularized objective
- **Next: turn the objective into policy updates**
 - Estimate gradients through sampled responses — **REINFORCE**
 - Stabilize the updates and reuse samples — **PPO**
 - Train directly from preference pairs — **DPO**

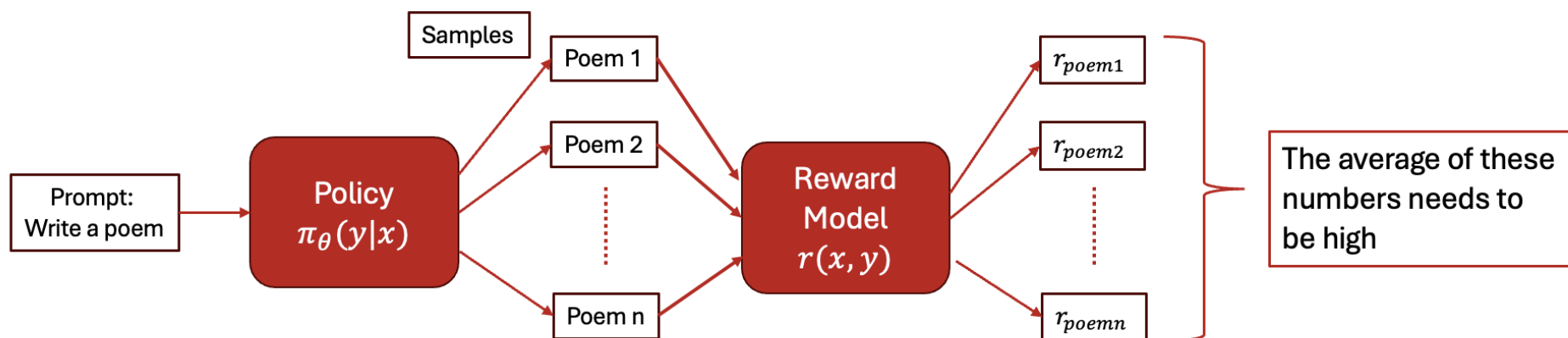


04

Training the Policy with REINFORCE

Can we just backpropagate?

$$J(\theta; x) = E_{y \sim \pi_{\theta}(y|x)} [r_{\phi}(x, y)]$$



Log-derivative trick

MOVE THE GRADIENT ONTO LOG PROBABILITY

$$\nabla_{\theta} \pi_{\theta}(y) = \pi_{\theta}(y) \nabla_{\theta} \log \pi_{\theta}(y)$$



$$\nabla_{\theta} E_{y \sim \pi_{\theta}} [r(x, y)] = E_{y \sim \pi_{\theta}} [r(x, y) \nabla_{\theta} \log \pi_{\theta}(y)]$$

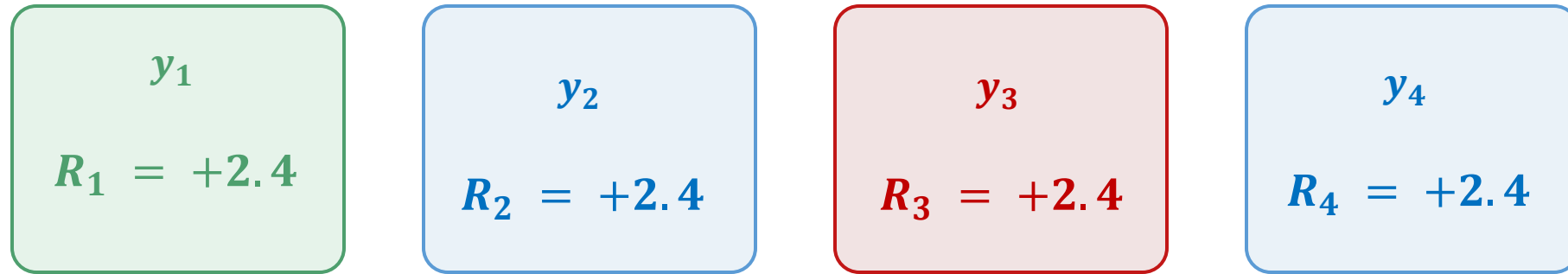
The reward can now weight a differentiable log-probability term.



REINFORCE

R_i is the total reward

SAMPLE A BATCH



$$\hat{g} = \left(\frac{1}{B} \right) \sum_i R_i \nabla_{\theta} \log \pi_{\theta}(y_i | x_i)$$

Batching averages independent Monte Carlo samples; it does not change the estimator's expectation.



A simple implementation

For each input x in batch

- 1 Generate $y \sim \pi_\theta(\cdot | x)$**
- 2 Score the completed response**
 $R = r(x, y)$ may come from RM or verifier
- 3 Recompute token log-probabilities**
 $logp = \sum_t \log \pi_\theta(y_t | x, y_{<t})$



A simple implementation

For each input x in batch

1 **Generate** $y \sim \pi_{\theta}(\cdot | x)$

2 **Score the completed response**
 $R = r(x, y)$ may come from RM or verifier

3 **Recompute token log-probabilities**
 $logp = \sum_t \log \pi_{\theta}(y_t | x, y_{<t})$

4 **Multiply, average and backpropagate**
 $(R * logp).mean().backward()$

with `no_grad()`:

```
Y = policy.generate(X)  
R = scorer(X, Y)
```

```
logp = policy.sequence_logprob(X, Y)  
loss = -(R * logp).mean()
```

```
loss.backward()  
optimizer.step()
```



Is REINFORCE enough?

Yes—in expectation, this is already a valid policy-gradient algorithm.

01

Generate

Sample a batch from the current policy.

02

Score

Reward each complete response.

03

Update

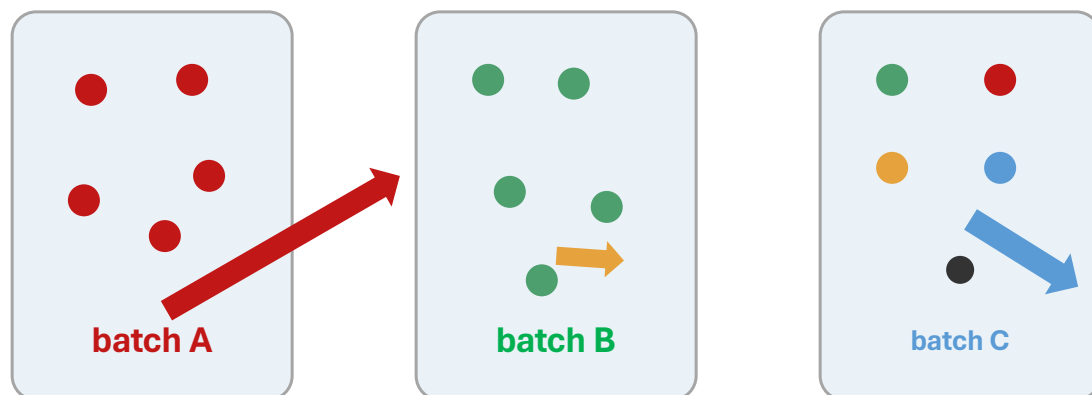
Apply the reward-weighted log-likelihood gradient.

Practically, it can be unstable and impractical



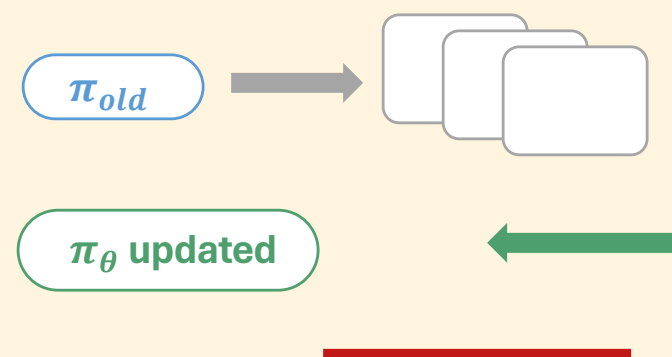
Why is REINFORCE unstable and impractical?

Same policy, different finite batches



The update direction changes drastically with the batch

Expensive responses
become stale



Old responses are never reused



We can compute the gradient. How do we make the update practical?

- **We can now turn rewards into policy gradients**
 - Discrete sampling prevents ordinary backpropagation
 - The log-derivative trick moves the gradient onto log-probabilities
 - REINFORCE uses sampled rewards to reinforce or suppress responses
 - In practice, finite-sample REINFORCE updates have high variance
- **Next: make policy optimization practical**
 - Reduce variance using a baseline and advantage
 - Reuse sampled responses using importance weighting
 - Limit excessive policy changes using PPO clipping
 - Then ask whether we can bypass the RL loop entirely — **DPO**



05

Variance Reduction & Batch Reuse

Proximal Policy Optimization

Addressing the issues

1 Variance

Can we reduce gradient estimate variation ?

2 Sample reuse

Can we reuse responses from the old policy



Baseline subtraction to reduce variance

CHANGE VARIANCE, NOT EXPECTATION

$$\hat{g}_b = (R - b(s)) \nabla_{\theta} \log \pi_{\theta}(y|x)$$

- **b may depend on the state—but not on the sampled action.**
- **Reasonable baselines**
 - Average reward over the batch.
 - Average reward over the group – GRPO
 - A learnt value function at each step – PPO

Prove that $E[\hat{g}_b] = E[\hat{g}]$



Baseline subtraction - Advantage

$$\hat{g}_b = (R - b(s)) \nabla_{\theta} \log \pi_{\theta}(y|x)$$



Baseline subtraction - Advantage

$$\hat{g}_b = (R - b(s)) \nabla_{\theta} \log \pi_{\theta}(y|x)$$

Not covered:

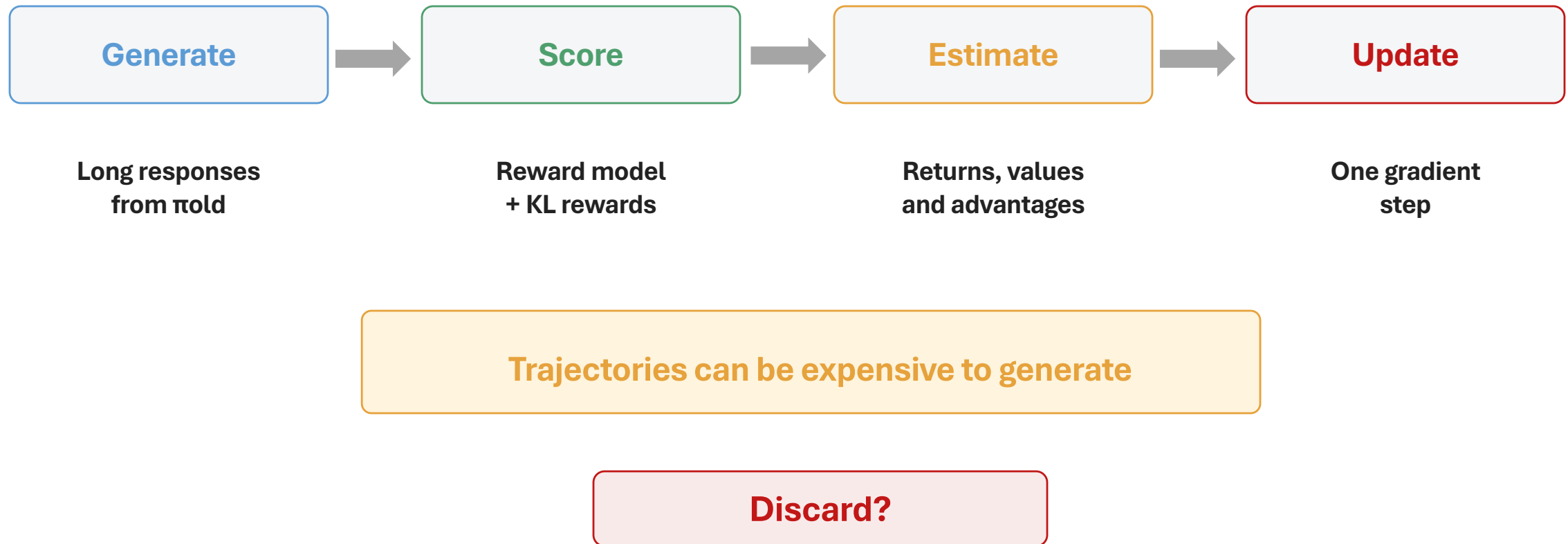
- Reward-to-go
- Value function estimation
- Token level advantages
- General advantage estimation

Variance reduction

What about batch reuse?



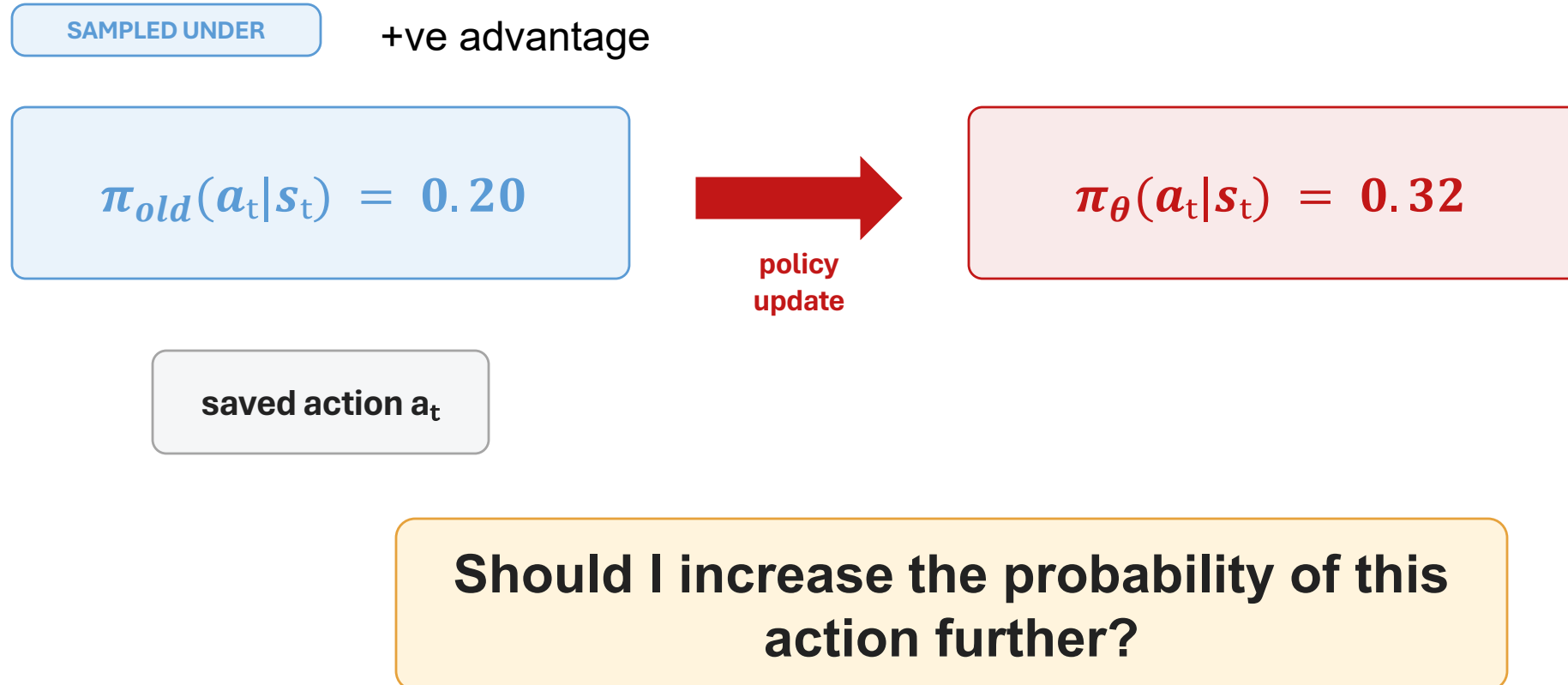
We paid for this batch. Must we discard it now?



The importance-sampling trick



Can I reuse these samples forever?



Bounding the updates allows reuse

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}$$

$A_t > 0$

Reward increasing probability



stop rewarding further
increase

$A_t < 0$

Reward decreasing probability



stop rewarding further
decrease



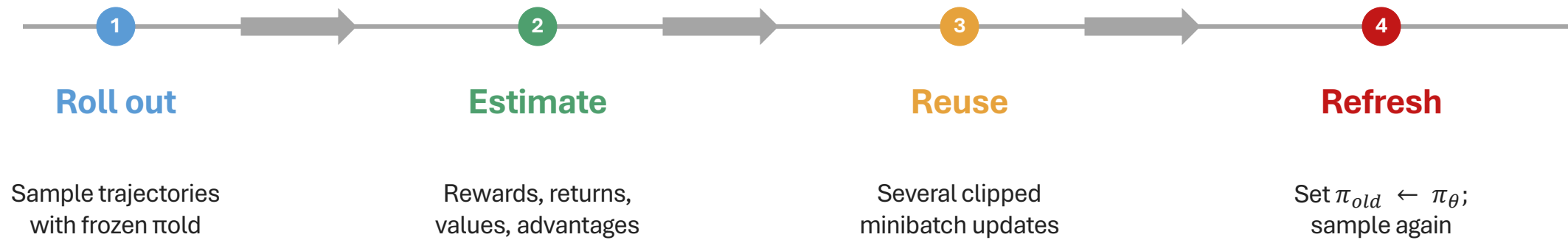
PPO objective

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}$$

$$L_{clip} = E_t[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$



The PPO algorithm



Do we still need the entire RL loop?

- **We can now optimize an explicit reward**
 - REINFORCE converts response rewards into policy gradients
 - Baselines and advantages reduce gradient variance
 - Importance ratios allow batch reuse
 - PPO clipping limits excessive policy changes
- **Next: return to the original preference pairs**
 - Can preference pairs train the policy directly?
 - Can we retain the reference-policy constraint?
 - Can we remove the explicit reward model and online RL loop? — **DPO**



06

Alignment without a reward model

Direct Preference Optimization

We began with preferences. Did we need all this?

Where we started

(x, y^+, y^-)

One human comparison

What we built

reward
model

online
rollouts

Advantage
computation

PPO
loop

Can the preference pair train π_θ directly while retaining the reference-policy constraint?



Why not train directly on chosen and rejected responses?

chosen response
 y^+



increase

rejected response
 y^-

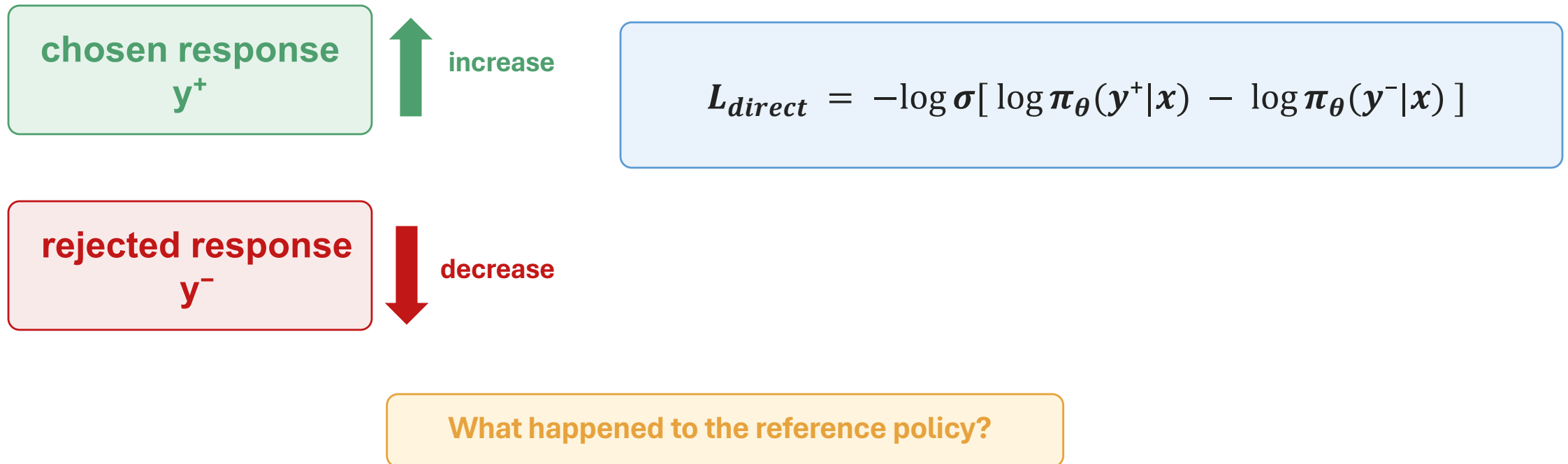


decrease

$$L_{direct} = -\log \sigma[\log \pi_{\theta}(y^+|x) - \log \pi_{\theta}(y^-|x)]$$



Why not train directly on chosen and rejected responses?



Direct ranking is easy. Preserving the reference-relative ranking is challenging



What would let us eliminate the reward model?

In the reward modelling objective

$$P(y^+ \succ y^- | x) = \sigma(r(x, y^+) - r(x, y^-))$$



replace

$$r(x, y^+) - r(x, y^-) = \text{function of } \pi_\theta \text{ and } \pi_{ref}$$

If the policy can supply this difference, the explicit reward model is no longer needed.



Finding a policy-based expression for the reward

$$\max_{\pi} E_{\pi}[r(x, y)] - \beta D_{KL}(\pi(\cdot | x) || \pi_{ref}(\cdot | x))$$

At its optimum, reward tilts the reference distribution

$$\pi^*(y|x) = \left[\frac{1}{Z(x)} \right] \pi_{ref}(y|x) \exp\left(\frac{r(x, y)}{\beta}\right)$$



rearrange

$$r(x, y) = \beta \log \left[\frac{\pi^*(y|x)}{\pi_{ref}(y|x)} \right] + \beta \log Z(x)$$



Finding a policy-based expression for the reward



Finding a policy-based expression for the reward



Reward difference in terms of π_θ and π_{ref}

$$r(x, y) = \beta \log \left[\frac{\pi^*(y|x)}{\pi_{ref}(y|x)} \right] + \beta \log Z(x)$$

Both responses share the same prompt x —and therefore the same $Z(x)$.

$$r(x, y^+) - r(x, y^-)$$



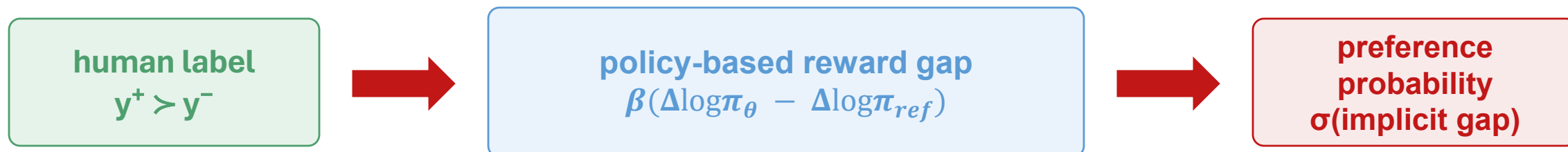
$$\beta \left\{ \log \left[\frac{\pi^*(y^+|x)}{\pi_{ref}(y^+|x)} \right] - \log \left[\frac{\pi^*(y^-|x)}{\pi_{ref}(y^-|x)} \right] \right\}$$

$$+ \beta \log Z(x) - \beta \log Z(x) = 0$$

Pairwise preferences need exactly the quantity for which the unknown normaliser cancels.



Training directly from the original preference pair



$$L_{DPO} = -E \log \sigma \left\{ \beta \left[\log \frac{\pi_\theta(y^+|x)}{\pi_{ref}(y^+|x)} - \log \frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)} \right] \right\}$$

Preference pairs train the policy directly.

No explicit reward model • no critic • no online rollout loop



Which route should we use?

Start with the feedback you can trust as the policy changes.

OFFLINE DPO

1

A strong, representative preference dataset already covers the target behaviour.

2

Compute and implementation simplicity are major constraints.

PPO / ONLINE RL

3

Newly generated responses can be scored reliably.

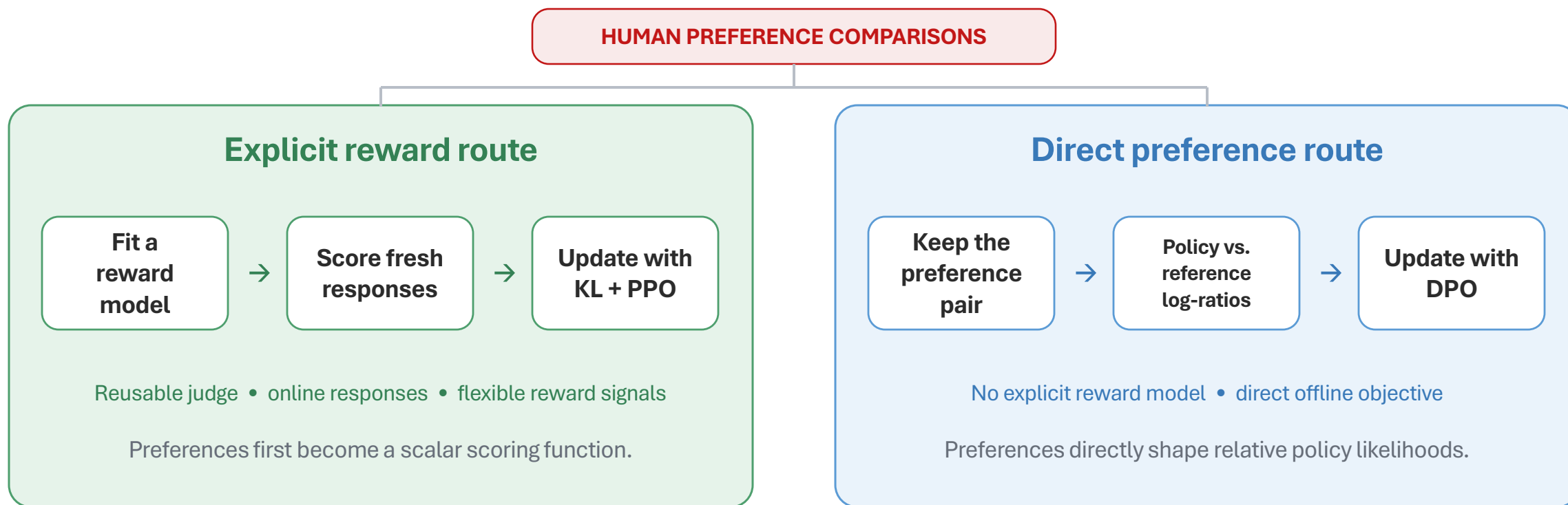
4

A trusted scalar reward or automatic verifier is available.

The feedback regime matters more than a universal PPO-versus-DPO ranking.



Two routes from human preferences to a better policy



**The algorithm determines how the model learns—
but the supervision signal determines what it learns.**

